Process Execution Engine

Low-Level Design Document

Class Skeletons · Method Contracts · Design Rationale

1. Purpose & Conventions

This document specifies the low-level class design for the Unifize Process Execution Engine. Each section covers one class: its single responsibility, its fields, and the contract of every method it exposes. Method bodies are intentionally omitted — signatures and contracts are the deliverable.

All classes are written in idiomatic Java. Every class owns exactly one concern. All dependencies are injected via constructor. No class reaches through another class's dependency. Checked exceptions are used for business-rule violations; unchecked for programming errors.

Convention	Return types use domain objects, never raw maps or primitives. Error conditions throw typed exceptions defined in section 21. Void methods that mutate state also write to the audit log inside the same transaction.

2. Annotation & Colour Legend

Every class section in this document opens with a colour-coded Responsibility annotation box. The colour is not decorative — it signals the architectural role of the class at a glance. This section defines the full legend so reviewers can navigate the document without ambiguity.

2.1 Responsibility Box Anatomy

Each box has two parts: a coloured label on the left and a plain-text description on the right.

Label *(colour = role)*	Single-sentence statement of what this class owns and what it does not own. Everything outside this statement belongs to a different class.

2.2 Colour Meanings

Swatch	Label Hex	Background Hex	Architectural Role	Classes Using This Colour
	#2E75B6	#D6E4F0	Value Objects & Orchestration	ProcessEngine, StateStore, EventPublisher, DefinitionRegistry (read path)
	#1A7A6E	#D4EFEC	Validation & Registry	DefinitionValidator, DefinitionRegistry, StepActivator
	#4B2E83	#EDE7F6	Mutable State & Lifecycle	ProcessInstance, StepState, InstanceFactory, ParallelQuorumChecker
	#8B1A1A	#FAE0E0	Guard & Enforcement	TransitionGuard, AuditWriter
	#7B4F00	#FFF3CC	Async & Scheduling	FieldValidator, EscalationScheduler, EscalationWorker

2.3 Other Annotation Types Used in This Document

Swatch	Label	When It Appears
	Convention	Documents a design rule that applies globally across all classes. Appears in Section 1 and where a pattern is first established.
	Design Note	Explains a deliberate architectural trade-off or a non-obvious decision. Appears inline where the choice is made, most notably in the Class Dependency Map.
	Edge Case	Flags a known runtime scenario that requires special handling. These map directly to the edge cases documented in the HLD (Sections 6 and 4).
	Responsibility	Opens every class section. States the single concern the class owns and, by omission, what it explicitly does not own.
	Guard	Highlights a hard enforcement boundary — a check that must never be bypassed. Appears on TransitionGuard and AuditWriter sections.

2.4 Code Block Convention

Dark-background monospaced blocks show class or enum skeletons only. They never contain method bodies — those are described in plain English in the method tables immediately below them. Purple text in method tables denotes a method signature. Teal text in field tables denotes a Java type.

Convention	All hex colour codes shown in this legend are the exact values used in the document's source. If this document is regenerated, these values must remain unchanged to preserve visual consistency across the PRD, HLD, and LLD trilogy.

3. Domain Types & Enums

These are pure value carriers. They hold no logic, no dependencies, and are immutable once constructed. They form the shared vocabulary for every class in the engine.

StepType (enum)

```
public enum StepType {
    TASK,               // actor completes a free-form work item
    APPROVAL,           // single actor approves or rejects
    PARALLEL_APPROVAL,  // multiple actors; quorum policy governs advancement
    REVIEW,             // actor reviews and submits comments; does not block on rejection
    CHECKLIST,          // actor marks discrete items done; all must be checked
    SIGNATURE           // actor provides a legally binding electronic signature
}
```

Drives the engine's dispatch logic. Each value maps to a distinct execution strategy.

Alignment Note	REVIEW is present in the original assignment's feature table alongside Approval, Task, Checklist, and Signature. It was absent from the first LLD version. REVIEW differs from APPROVAL in that a rejection by the reviewer routes back with comments but does not hard-block the process — the routing rule decides the consequence, not the step type itself.

StepStatus (enum)

```
public enum StepStatus {
    PENDING,           // created, not yet started
    IN_PROGRESS,       // actor has opened the step
    COMPLETED,         // terminal — positive outcome
    REJECTED,          // terminal — negative outcome, may trigger re-route
    ESCALATED,         // terminal — threshold exceeded, re-assigned
    SKIPPED            // terminal — conditional routing bypassed this step
}
```

Terminal statuses are never overwritten. A new StepState record is created if a step is revisited after a rejection loop.

ActionType (enum)

```
public enum ActionType {
    OPEN,              // actor explicitly opens a PENDING step → IN_PROGRESS
    SUBMIT,            // actor submits fields and advances
    APPROVE,           // actor approves an approval step
    REJECT,            // actor rejects — triggers routing re-evaluation
    ESCALATE,          // system-initiated; actor = SYSTEM_USER
    REASSIGN,          // admin re-assigns an open step
    REOPEN             // QA Manager re-opens after effectiveness failure
}
```

Action (Immutable Value Object)

The original assignment defines Action as a first-class domain type passed into AdvanceStep. Rather than dissolving it into a loose ActionType enum plus a raw Map, we model it as an explicit value object. This keeps the engine interface clean and makes the intent of each call self-documenting.

```
public final class Action {

    private final ActionType        type;    // what the actor is doing
    private final Map<String,Object> fields; // submitted field values for this step
    private final String            reason;  // optional comment (required on REJECT)

    // Static factory methods — prevents raw construction
    public static Action approve()                              { ... }
    public static Action reject(String reason)                  { ... }
    public static Action submit(Map<String,Object> fields)      { ... }
    public static Action escalate(String reason)                { ... }
    public static Action reopen(String reason)                  { ... }

    // Accessors only — no setters
    public ActionType         getType()   { ... }
    public Map<String,Object> getFields() { ... }
    public String             getReason() { ... }
}
```

Alignment Note The original assignment declares: AdvanceStep(instanceID, stepID, action Action, actor User). Our engine interface mirrors this exactly. The Action object encapsulates both the ActionType and submitted fields, so advanceStep does not need a separate fields parameter.

StepResult (Immutable Value Object)

StepResult is the return type of advanceStep. It was referenced but never defined in the first LLD version. It carries everything the caller needs to understand what happened as a result of their action — without requiring a second query.

```
public final class StepResult {

    private final String          instanceId;
    private final String          completedStepId;   // the step that just transitioned
    private final StepStatus      outcomeStatus;     // terminal status reached
    private final List<String>    nextStepIds;       // step(s) now active; empty if process
done
    private final boolean         processCompleted;  // true if instance reached terminal
state
    private final List<String>    remainingActors;   // for parallel steps: who still needs to
act
    private final AuditEntry      auditEntry;        // the audit record written for this
action

    // Accessors only — no setters
}
```

Field	Type	Purpose
`instanceId`	`String`	ID of the ProcessInstance that was advanced
`completedStepId`	`String`	The stepId that just reached a terminal status
`outcomeStatus`	`StepStatus`	The terminal status the step moved to: COMPLETED, REJECTED, ESCALATED
`nextStepIds`	`List<String>`	The step IDs now active. Multiple if parallel steps were activated. Empty if the process has completed.
`processCompleted`	`boolean`	True when the instance has reached its final step and is now COMPLETED. Allows callers to react without polling.
`remainingActors`	`List<String>`	For PARALLEL_APPROVAL steps: user IDs of participants who have not yet acted. Empty for non-parallel steps.

`auditEntry`	`AuditEntry`	The AuditEntry written for this specific action. Returned so callers can log or forward the record without a separate read.

Alignment Note	The original assignment specifies AdvanceStep returns StepResult. This definition closes the gap identified in the alignment review. StepResult is read-only — it is a snapshot of what happened, not a handle to further mutate state.

UserResolver (Interface)

UserResolver is a boundary interface between the process engine and the user directory. The engine never constructs User objects from raw IDs. All User hydration — resolving a display name, email, or role from an ID — goes through this interface. The implementation is outside engine scope (LDAP, database, or identity service).

```
public interface UserResolver {

    // Resolve a single user by ID. Throws UserNotFoundException if not found.
    User resolve(String userId);

    // Resolve a batch of user IDs in one call. Avoids N individual lookups.
    Map<String, User> resolveAll(Collection<String> userIds);

    // Resolve all users that carry the given role. Used by EscalationPolicy
    // to find the escalation target without hardcoding a specific user ID.
    List<User> resolveByRole(String role);
}
```

Design Note	UserResolver is injected into StepActivator and EscalationWorker. It is the fix for the escalation display name bug (ProcessEngine.java:195): new User(assignedTo, assignedTo) is replaced with userResolver.resolve(assignedTo). The interface also supports resolveByRole() so EscalationPolicy can declare a role (e.g. 'QA_MANAGER') rather than a hardcoded user ID.
Edge Case	UserResolver.resolve() must never be called inside a database transaction — user directory latency would hold a row lock. Resolve all users before opening the transaction, then pass the resolved User objects in.

4. ProcessDefinition (Immutable Value Object)

Responsibility	Carries the complete, versioned blueprint of a workflow. Once constructed it never changes. It does not validate itself — that is DefinitionValidator's job.

Field	Type	Purpose
`definitionId`	`String`	Globally unique stable identifier across all versions
`version`	`int`	Monotonically increasing. Starts at 1. Never reused.
`name`	`String`	Human-readable process name (e.g. 'CAPA Process')
`steps`	`List<StepDefinition>`	Ordered list of step blueprints. Order determines default flow.
`routingRules`	`List<RoutingRule>`	Outbound transition conditions evaluated after each step completes
`publishedAt`	`Instant`	Server-assigned timestamp when this version was published
`publishedBy`	`String`	User ID of the administrator who published this version

5. StepDefinition (Immutable Value Object)

Responsibility	Describes one node in the process graph. Carries all static constraints — field rules, quorum policy, escalation threshold, assignee expression. Does not execute anything.

Field	Type	Purpose
`stepId`	`String`	Unique within the ProcessDefinition version
`stepType`	`StepType`	Determines which execution strategy the engine dispatches to
`assigneeRule`	`AssigneeRule`	Expression resolving to one or more users at runtime
`fieldSchemas`	`List<FieldSchema>`	Declares each input field: name, type, required flag, validation regex
`validationRules`	`List<ValidationRule>`	Cross-field business rules evaluated as a group before transition
`quorumPolicy`	`QuorumPolicy`	Applicable only to PARALLEL_APPROVAL steps. Defines ALL or MAJORITY.
`escalationPolicy`	`EscalationPolicy`	Optional. Defines threshold duration and escalation target role.
`scheduledOffset`	`Duration`	Optional. For scheduled steps (e.g. effectiveness check at +30 days).

FieldSchema (Nested Value Object within StepDefinition)

FieldSchema describes one input field on a step. It carries both the raw regex string and its pre-compiled Pattern. Compilation happens exactly once — at definition registration time in DefinitionValidator — not on the validation hot path.

Field	Type	Purpose
`name`	`String`	Field name — matches the key in the submitted fields map
`type`	`FieldType`	STRING | NUMBER | DATE | BOOLEAN — drives type coercion before validation
`required`	`boolean`	If true, FieldValidator rejects the submission when this field is absent or blank
`regex`	`String`	Raw regex pattern as authored in the no-code builder. Stored for serialisation and display.
`compiledPattern`	`Pattern`	Pre-compiled form of regex. Null when no regex is declared. Set once by DefinitionValidator.compilePatterns() at registration time. Never recompiled at runtime.

Edge Case	compiledPattern is transient with respect to serialisation — it is not stored in the database. When a ProcessDefinition is loaded from the registry, DefinitionRegistry.register() re-compiles all patterns before returning the object. This keeps the hot path (FieldValidator.checkRegexPatterns) to a single Pattern.matcher() call with no allocation.

6. ProcessInstance (Mutable Aggregate Root)

Responsibility	Represents one running execution. Holds the pinned definition version reference and the current runtime state. This is the only mutable entity. Exposes only state-reading methods — all mutations happen through the engine, never directly.

Field	Type	Purpose

instanceId	String	Globally unique ID assigned at StartProcess time
definitionId	String	FK to the ProcessDefinition this instance was started on
definitionVersion	int	Pinned at creation. Never changes. Immutable contract to HLD E2.
initiator	User	The user who started this process instance
activeStepIds	Set<String>	Current set of step IDs awaiting action. Multiple if parallel steps are open.
fieldValues	Map<String,Object>	Accumulated field values across all steps completed so far
status	InstanceStatus	ACTIVE | COMPLETED | CANCELLED | SUSPENDED
version	long	Optimistic lock counter. Incremented on every state mutation.
createdAt	Instant	Server-assigned creation timestamp

Method Signature	Responsibility
getPinnedDefinitionVersion() : int	Returns the immutable definition version this instance is bound to. Called by the engine before every operation to load the correct ProcessDefinition.
getActiveStepIds() : Set<String>	Returns the set of step IDs currently awaiting action. Used by the engine to validate that an incoming AdvanceStep targets a currently open step.
getFieldValue(String fieldName) : Object	Returns the current value of one field. Used by RoutingRuleEvaluator to evaluate conditional routing expressions.
isActive() : boolean	Returns true if status is ACTIVE. Guards against any operation being applied to a completed or cancelled instance.
getLockVersion() : long	Returns the current optimistic lock counter. Passed to the StateStore for conflict detection on write.

7. StepState (Immutable Audit-Friendly Record)

Responsibility	Tracks the execution lifecycle of one StepDefinition within one ProcessInstance. For PARALLEL_APPROVAL steps, one StepState record exists per participant. StepState records are never updated — a terminal status is written as a new field value; the record is otherwise frozen.

Field	Type	Purpose
stepStateId	String	Globally unique ID
instanceId	String	FK to owning ProcessInstance
stepId	String	FK to StepDefinition within the pinned definition version
assignedTo	String	User ID resolved at step activation time
status	StepStatus	Current lifecycle status of this step execution. Starts as PENDING, never skips directly to IN_PROGRESS.
createdAt	Instant	Server-assigned timestamp when this StepState was created and status was PENDING. Start of the time-to-open SLA window. Never null.
openedAt	Instant	Server-assigned timestamp when the assigned actor called openStep and status moved to IN_PROGRESS. Null until openStep is called. Enables time-to-open vs time-to-complete SLA tracking as distinct measurements.
completedAt	Instant	Server-assigned timestamp when status reached a terminal value. Null if still open.

`participantId`	`String`	Populated only for PARALLEL_APPROVAL rows. Identifies the specific participant.
`sequenceNumber`	`long`	Monotonic sequence for deterministic ordering independent of wall clock.

Edge Case	PENDING and IN_PROGRESS are not cosmetic. PENDING = step exists and actor is assigned but has not acknowledged it. IN_PROGRESS = actor has opened it and the completion SLA clock has started. getRemainingParticipants() must include both PENDING and IN_PROGRESS participants — excluding PENDING would silently undercount outstanding actors on a parallel step.

8. ProcessEngine (Orchestration Facade)

Responsibility	The single public entry point into the engine. It owns no business logic. Its only job is to sequence calls to specialist collaborators in the correct order and handle the transaction boundary. Thin by design.

Constructor dependencies injected:

```
public class ProcessEngine {

    private final DefinitionValidator    definitionValidator;
    private final DefinitionRegistry     definitionRegistry;
    private final InstanceFactory        instanceFactory;
    private final StepActivator          stepActivator;
    private final TransitionGuard        transitionGuard;
    private final FieldValidator         fieldValidator;
    private final RoutingRuleEvaluator   routingRuleEvaluator;
    private final ParallelQuorumChecker  quorumChecker;
    private final AuditWriter            auditWriter;
    private final StateStore             stateStore;
    private final EscalationScheduler    escalationScheduler;
    private final EventPublisher         eventPublisher;

    // Constructor omitted — all fields final, injected
}
```

Method Signature	Responsibility
`loadDefinition(ProcessDefinition def) : void`	Delegates to DefinitionValidator to assert structural correctness and absence of routing cycles. On success, registers the definition via DefinitionRegistry. Throws DefinitionValidationException on any violation.
`startProcess(String definitionId, User initiator, Map<String,Object> fields) : ProcessInstance`	Resolves the latest published version via DefinitionRegistry. Delegates to FieldValidator for Step 1 fields. Delegates to InstanceFactory to construct the ProcessInstance. Activates Step 1 via StepActivator — Step 1 is created with status PENDING. Writes initial AuditEntry via AuditWriter. Persists via StateStore. Returns the constructed instance.
`openStep(String instanceId, String stepId, User actor) : void`	Transitions a PENDING StepState to IN_PROGRESS for the given actor. Records the openedAt timestamp. Writes an AuditEntry with ActionType.OPEN. This call is required before advanceStep can be called — a step cannot be advanced while still PENDING. Enforced by TransitionGuard.assertStepIsInProgress().
`advanceStep(String instanceId, String stepId, Action action, User actor) : StepResult`	Mirrors the original assignment's interface contract exactly. Loads the instance. Calls TransitionGuard to confirm the transition is legal. Delegates field validation to FieldValidator via action.getFields(). Applies the action. Evaluates routing via RoutingRuleEvaluator. Activates the

	next step(s) via StepActivator. Writes AuditEntry. Persists atomically via StateStore. Returns a populated StepResult.
`getAuditTrail(String instanceId) : List<AuditEntry>`	Delegates to AuditWriter's read path. Returns AuditEntries ordered by sequenceNumber ascending. Performs no business logic.

Alignment Note	advanceStep now accepts Action as a single value object — matching the original assignment's interface contract: AdvanceStep(instanceID, stepID, action Action, actor User). The Action class is defined in Section 3.
Design Note	openStep is a new method on the public interface, added to resolve the deferred PENDING lifecycle bug (DefaultStepActivator.java:37). activateStep now creates StepState with status PENDING. The actor must call openStep before advanceStep — this separation records the time-to-open timestamp and makes the PENDING state reachable and meaningful.
Design Note	The original assignment states 'in-memory data structures are sufficient'. This LLD introduces StateStore backed by persistent storage, which is a deliberate architectural upgrade beyond the assignment's minimum scope. The reason: in-memory storage cannot satisfy the immutability and auditability constraints at production scale. This deviation is intentional and documented here.

9. DefinitionValidator (Validation — Write Path)

Responsibility	Validates a ProcessDefinition before it is registered. Checks structural completeness and detects routing graph anomalies. Stateless — holds no fields. Every method is pure.

Method Signature	Responsibility
`validate(ProcessDefinition def) : void`	Orchestrates all validation sub-checks below in sequence. Throws DefinitionValidationException aggregating all errors found. Called exclusively by ProcessEngine.loadDefinition.
`compilePatterns(ProcessDefinition def) : void`	Iterates every FieldSchema across all StepDefinitions. For each schema that declares a non-null regex, calls Pattern.compile() once and sets schema.compiledPattern. Called by validate() after structural checks pass — only well-formed regex strings reach this step. This is the only point in the system where Pattern.compile() is called. FieldValidator.checkRegexPatterns() uses schema.compiledPattern directly with no recompilation.
`assertNoDuplicateStepIds(ProcessDefinition def) : void`	Scans stepId values across all StepDefinitions. Throws if any two share the same ID. Duplicate IDs make routing expressions ambiguous.
`assertAllRoutingTargetsExist(ProcessDefinition def) : void`	For every RoutingRule, verifies its target stepId is present in the definition's step list. Prevents routing to a non-existent step.
`assertEveryStepHasExitPath(ProcessDefinition def) : void`	Ensures every non-terminal StepDefinition has at least one outbound RoutingRule or a declared default successor. A step with no exit path would permanently stall any instance that reaches it.
`assertNoCyclicDependency(ProcessDefinition def) : void`	Performs DFS on the routing graph. Detects back-edges. Throws if a cycle is found. This is the guard against the circular cross-process dependency edge case (HLD E3).

`assertParallelStepsHaveQuorumPolicy(ProcessDefinition def) : void`	For every StepDefinition with type PARALLEL_APPROVAL, asserts that a non-null QuorumPolicy is declared. Rejects ambiguous parallel steps.

10. DefinitionRegistry (Definition Storage & Lookup)

Responsibility	Stores validated ProcessDefinitions and resolves them by ID and version. Maintains an LRU in-memory cache bounded by a configurable size to avoid reloading frequently used definitions from the store.

Method Signature	Responsibility
`register(ProcessDefinition def) : void`	Persists the definition to durable store and inserts it into the LRU cache. Assigns the next version number if the definitionId already exists. Idempotent for the same version. Precondition: def.compiledPatterns must already be populated by DefinitionValidator.compilePatterns() — register() does not compile patterns itself.
`resolveLatest(String definitionId) : ProcessDefinition`	Returns the highest published version for the given definitionId from cache or store. If loaded from store (cache miss), calls DefinitionValidator.compilePatterns() before returning — compiled patterns are transient and must be re-hydrated after deserialisation. Used by ProcessEngine.startProcess.
`resolveVersion(String definitionId, int version) : ProcessDefinition`	Returns the exact version requested from cache or store. On cache miss, re-compiles patterns before returning. Used by ProcessEngine.advanceStep to load the version pinned on the ProcessInstance. This is the mechanism that implements process versioning safety (HLD E2).
`evictFromCache(String definitionId) : void`	Removes all cached versions for a definitionId. Called after a new version is registered to force a fresh load — and therefore a fresh pattern compilation — on next access.

11. InstanceFactory (Instance Construction)

Responsibility	Constructs a new ProcessInstance from a definition and initiator. Assigns a globally unique instanceId and server timestamp. Sets the pinned definition version. Does not persist — only constructs.

Method Signature	Responsibility
`createInstance(ProcessDefinition def, User initiator, Map<String,Object> initialFields) : ProcessInstance`	Allocates a new instanceId (UUID). Pins the definition version from def.getVersion(). Sets status to ACTIVE. Copies initialFields into the instance field map. Sets version counter to 1. Returns the unpersisted ProcessInstance.

12. TransitionGuard (Transition Pre-Condition Enforcement)

Responsibility	The single gatekeeper for all state transitions. Checks every precondition before any write occurs. Stateless and pure — all information it needs is passed as arguments. Throws typed exceptions; never returns error codes.

Method Signature	Responsibility
`assertStepIsActive(ProcessInstance instance, String stepId) : void`	Verifies the stepId is in the instance's activeStepIds set. Rejects transitions targeting steps that are not currently open — guards against stale UI submissions.
`assertStepIsInProgress(StepState stepState) : void`	Verifies the StepState has status IN_PROGRESS before allowing advanceStep. A PENDING step cannot be advanced — the actor must call openStep first. This guard enforces the PENDING → IN_PROGRESS → terminal lifecycle and makes PENDING a reachable, meaningful state.
`assertActorIsAuthorised(StepDefinition stepDef, User actor) : void`	Evaluates the step's AssigneeRule against the acting User. Throws UnauthorisedTransitionException if the actor does not satisfy the rule. Prevents wrong-user actions.
`assertInstanceIsActive(ProcessInstance instance) : void`	Checks instance.isActive(). Throws InactiveInstanceException if the instance is completed or cancelled. Guards all mutation paths.
`assertActionIsValidForStepType(StepType stepType, Action action) : void`	Validates that action.getType() is legal for the StepType (e.g. APPROVE is only valid on APPROVAL, PARALLEL_APPROVAL, and REVIEW steps). Prevents semantically invalid operations. Accepts the full Action object so it can also assert that required fields like reason are present for REJECT actions.
`assertOptimisticLockVersion(ProcessInstance instance, long expectedVersion) : void`	Compares the caller's expected version with the instance's stored version. Throws OptimisticLockException if they differ. This is the concurrency guard for HLD E1.

13. FieldValidator (Field & Business Rule Validation)

Responsibility	Validates submitted field values against a StepDefinition's FieldSchemas and ValidationRules. Stateless. Returns a typed validation result rather than throwing, so the caller can accumulate and return all errors at once.

Method Signature	Responsibility
`validate(StepDefinition stepDef, Action action) : ValidationResult`	Orchestrates all field-level and cross-field checks against action.getFields(). Returns a ValidationResult containing the full list of violations. Empty result means valid. Accepts the full Action so it can also check action-type-specific requirements (e.g. REJECT requires a non-blank reason).
`checkRequiredFields(StepDefinition stepDef, Map<String,Object> fields) : List<FieldViolation>`	Iterates fieldSchemas, finds all fields with required=true that are absent or blank in the submitted map. Returns one FieldViolation per missing field.
`checkRegexPatterns(StepDefinition stepDef, Map<String,Object> fields) : List<FieldViolation>`	For each FieldSchema where compiledPattern is non-null, calls schema.compiledPattern.matcher(value).matches() directly. No Pattern.compile() call here — the pattern was compiled once at definition registration time by DefinitionValidator.compilePatterns(). Skips blank values (required check is separate). This is a hot path: zero allocation beyond the Matcher object.
`checkCrossFieldRules(StepDefinition stepDef, Map<String,Object> fields) : List<FieldViolation>`	Evaluates each ValidationRule expression against the full field map. Example: 'if severity=Critical then root_cause_analysis is required'. Returns violations for unsatisfied rules.

14. RoutingRuleEvaluator (Conditional Routing)

Responsibility	Determines the next step(s) after a step completes, by evaluating the process definition's routing rules against the current instance field values. Stateless and pure.

Method Signature	Responsibility
`resolveNextSteps(ProcessDefinition def, String completedStepId, ProcessInstance instance) : List<String>`	Finds all RoutingRules whose sourceStepId equals completedStepId, evaluates their conditions in declaration order against instance field values, returns the stepId(s) from the first matching rule. Falls back to the default route if no condition matches. Throws NoRoutingMatchException if no rule matches and no default exists — this indicates a malformed definition that slipped through validation.
`evaluateCondition(RoutingRule rule, Map<String,Object> fieldValues) : boolean`	Evaluates one RoutingRule's boolean condition expression against the provided field values. Single-purpose: one rule, one result. Uses a sandboxed expression evaluator to prevent injection.

15. StepActivator (Step Lifecycle Activation)

Responsibility	Creates StepState records when a step is opened. Resolves the concrete assignee from the AssigneeRule. Schedules escalation timers when applicable. Does not write to the audit log — that is AuditWriter's job.

Method Signature	Responsibility
`activateStep(ProcessInstance instance, StepDefinition stepDef) : List<StepState>`	Creates one StepState per assignee with status PENDING — never IN_PROGRESS. The PENDING state records the createdAt timestamp and starts the time-to-open SLA clock. For PARALLEL_APPROVAL steps, creates one PENDING StepState per participant. Calls resolveAssignees and scheduleEscalationIfRequired. Returns all created records for ProcessEngine to persist.
`resolveAssignees(StepDefinition stepDef, ProcessInstance instance) : List<User>`	Evaluates the StepDefinition's AssigneeRule against the ProcessInstance context. Returns fully resolved User objects — not raw IDs. Delegates to UserResolver to hydrate display names and metadata. For parallel steps, returns multiple Users — one StepState will be created per User.
`scheduleEscalationIfRequired(StepState stepState, EscalationPolicy policy) : void`	If the StepDefinition declares an EscalationPolicy, emits a delayed escalation event to the EscalationScheduler for each newly created StepState. Does nothing if no policy is declared.

Design Note	resolveAssignees now returns List<User> instead of List<String>. This closes the escalation display name bug (ProcessEngine.java:195) — the User object carries both the ID and the display name. Callers that previously constructed new User(id, id) must now call UserResolver instead. UserResolver is defined in Section 3.

16. ParallelQuorumChecker (Parallel Approval Evaluation)

Responsibility	Determines whether a PARALLEL_APPROVAL step's quorum has been satisfied after each participant acts. Completely separate from routing — routing only starts once this class confirms the quorum is met.

Method Signature	Responsibility
`isQuorumSatisfied(QuorumPolicy policy, List<StepState> participantStates) : boolean`	Counts completed vs total participant StepStates. For ALL policy: returns true only when every participant has reached COMPLETED. For MAJORITY policy: returns true when more than half have completed. Called after each individual participant's AdvanceStep.
`hasAnyRejection(List<StepState> participantStates) : boolean`	Returns true if any participant StepState has status REJECTED. If true, the engine bypasses quorum and routes back to the prior step with accumulated rejection comments. Rejection by one blocks the whole parallel group.
`getRemainingParticipants(List<StepState> participantStates) : List<String>`	Returns the user IDs of participants whose StepState is still IN_PROGRESS or PENDING. Used by the engine to construct the StepResult payload, informing callers who still needs to act.

17. AuditWriter (Immutable Audit Log)

Responsibility	The sole class permitted to write AuditEntry records. Also owns the read path. Enforces append-only semantics — no update or delete method exists on this class at all.

Field	Type	Purpose
`sequenceGenerator`	`SequenceGenerator`	Provides monotonically increasing sequence numbers independent of wall clock. Shared across all writers.

Method Signature	Responsibility
`record(String instanceId, String stepStateId, User actor, StepStatus fromStatus, StepStatus toStatus, Action action) : AuditEntry`	Constructs a new AuditEntry with a server-assigned UTC timestamp and the next sequence number from the generator. Stores actor, fromStatus, toStatus, action.getType(), and action.getReason() as the reason field. Persists immediately. Returns the created entry. This is the only write path to the audit table.
`recordSystemAction(String instanceId, String stepStateId, ActionType action, String reason) : AuditEntry`	Convenience overload for system-initiated events (escalations, scheduled triggers). Substitutes SYSTEM_USER as the actor. Delegates to record(). Explicit method prevents accidental use of a null actor.
`getTrailForInstance(String instanceId) : List<AuditEntry>`	Returns all AuditEntries for the given instanceId, ordered by sequenceNumber ascending. Used by ProcessEngine.getAuditTrail. Never filters, never paginates — returns the complete ordered log.
`getEntriesByDateRange(Instant from, Instant to) : List<AuditEntry>`	Returns all AuditEntries with timestamps in the given range, across all instances. Used for compliance reports. Operates against the read model / reporting partition, not the primary write table.

18. StateStore (Atomic Persistence)

Responsibility	Persists ProcessInstance and StepState mutations. Enforces optimistic locking. Guarantees that the instance state update and its AuditEntry are written in a single atomic operation. This class is the only place that talks to the database.

Method Signature	Responsibility

`saveInstance(ProcessInstance instance) : void`	Persists a new ProcessInstance record. Sets the initial lock version. Throws DuplicateInstanceException if the instanceId already exists.
`updateInstance(ProcessInstance instance, List<StepState> stepStates, AuditEntry auditEntry) : void`	The core atomic write operation. Within a single transaction: updates the ProcessInstance (including incrementing the lock version), upserts the provided StepState records, and inserts the AuditEntry. Throws OptimisticLockException if the stored lock version differs from the instance's version. If any part fails, the entire transaction rolls back.
`loadInstance(String instanceId) : ProcessInstance`	Loads a ProcessInstance by ID including its current lock version. Throws InstanceNotFoundException if absent.
`loadStepStates(String instanceId) : List<StepState>`	Loads all StepState records for a given instanceId. Used by ParallelQuorumChecker and the audit read path.
`loadStepStatesForStep(String instanceId, String stepId) : List<StepState>`	Loads StepState records filtered to a specific stepId within an instance. Used by ParallelQuorumChecker when evaluating one specific parallel step's quorum.

19. EscalationScheduler (Async Escalation Dispatch)

Responsibility	Schedules future escalation events by publishing delayed messages to a durable queue. Does not fire escalations itself — that is EscalationWorker's job. Completely decoupled from the main execution path.

Method Signature	Responsibility
`scheduleEscalation(StepState stepState, EscalationPolicy policy) : void`	Publishes an EscalationEvent message to the durable queue with a delivery delay equal to policy.getThreshold(). The message contains the instanceId, stepStateId, and expected current status so the worker can verify freshness on consumption.
`cancelEscalation(String stepStateId) : void`	Attempts to cancel a pending escalation for the given stepStateId. Called when a step completes before its threshold expires. Cancellation is best-effort — if the message has already been delivered, the EscalationWorker's idempotency check will discard it.

20. EscalationWorker (Escalation Consumer)

Responsibility	Consumes EscalationEvents from the queue. Verifies the target step is still open. Calls ProcessEngine.advanceStep with ActionType.ESCALATE and actor=SYSTEM_USER. Idempotent — duplicate deliveries are silently discarded.

Method Signature	Responsibility
`handleEscalationEvent(EscalationEvent event) : void`	Entry point for the message consumer. Delegates to validateEscalationIsStillApplicable, and if valid, calls processEscalation.
`validateEscalationIsStillApplicable(EscalationEvent event) : boolean`	Loads the ProcessInstance. Returns false (discard) if the instance is not ACTIVE, or if the target StepState is no longer IN_PROGRESS. This is the idempotency guard and the fix for HLD E5 (escalation firing on a closed instance).
`processEscalation(EscalationEvent event) : void`	Invokes ProcessEngine.advanceStep with ActionType.ESCALATE, the SYSTEM_USER actor, and the reason from the EscalationPolicy. The engine

	writes the audit record and re-assigns the step per the escalation target in the policy.

21. EventPublisher (Domain Event Emission)

Responsibility	Publishes domain events to the message bus after successful state transitions. One-way outbound only. No return values. Failure to publish must not fail the main transaction — events are fire-and-forget with best-effort delivery.

Method Signature	Responsibility
`publishProcessStarted(ProcessInstance instance) : void`	Emits a ProcessStartedEvent after a new instance is successfully persisted. Consumed by the Notification Service.
`publishStepTransitioned(StepState stepState, Action action, User actor) : void`	Emits a StepTransitionedEvent for every step status change. Includes action.getType() and action.getReason() in the event payload. Consumed by Notification Service and downstream analytics consumers.
`publishProcessCompleted(ProcessInstance instance) : void`	Emits a ProcessCompletedEvent. Consumed by the Notification Service and any cross-process dependency listeners.
`publishEscalationTriggered(StepState stepState, User escalatedTo) : void`	Emits a dedicated EscalationTriggeredEvent. Consumed by the Notification Service for urgent alerting.

22. Exception Catalogue

All exceptions are checked unless noted. Each carries a machine-readable errorCode for API response mapping and a human-readable message for logging.

Exception Class	Type	When Thrown
`DefinitionValidationException`	Checked	LoadDefinition: structural errors or routing cycle detected
`UnauthorisedTransitionException`	Checked	AdvanceStep: actor does not satisfy the step's assignee rule
`InactiveInstanceException`	Checked	AdvanceStep: target instance is not in ACTIVE status
`InvalidTransitionException`	Checked	AdvanceStep: action is not valid for the step's current state or type
`OptimisticLockException`	Checked	StateStore: concurrent modification detected via version mismatch
`FieldValidationException`	Checked	AdvanceStep: submitted fields fail required, regex, or cross-field rules
`InstanceNotFoundException`	Unchecked	StateStore: instanceId does not exist — programming error in caller
`NoRoutingMatchException`	Unchecked	RoutingRuleEvaluator: no rule matched and no default — malformed definition
`DuplicateInstanceException`	Unchecked	StateStore: instanceId collision — should never occur with UUID generation

23. Class Dependency Map

Reading guide: A → B means A holds a direct reference to B via constructor injection. No class may hold a reference not shown here.

```
ProcessEngine
    ├── DefinitionValidator
    ├── DefinitionRegistry
    ├── InstanceFactory
    ├── TransitionGuard
    ├── FieldValidator
    ├── RoutingRuleEvaluator
    ├── StepActivator
    │       ├── UserResolver      (resolves assignee IDs to full User objects)
    │       └── EscalationScheduler
    ├── ParallelQuorumChecker
    ├── AuditWriter
    │       └── SequenceGenerator
    ├── StateStore
    └── EventPublisher

EscalationWorker
    ├── ProcessEngine   (calls advanceStep with SYSTEM_USER)
    ├── StateStore      (read-only — loads instance to validate before acting)
    └── UserResolver    (resolves escalation target ID to full User for event payload)

Value Objects (no dependencies — passed between classes as data)
    Action      — wraps ActionType + fields + reason
    StepResult  — wraps outcome of advanceStep; read-only snapshot

Interfaces (implemented outside engine scope)
    UserResolver — implemented by user directory / identity service adapter
```

Design Note	EscalationWorker depends on ProcessEngine for the mutation path. ProcessEngine does NOT depend on EscalationWorker. This one-way dependency prevents circular references and keeps the core engine unaware of the scheduling infrastructure.

24. Alignment with Original Assignment

This section documents every gap identified between the first LLD version and the original assignment's interface contract, and confirms how each has been addressed in this version.

	Gap	Original Contract	Previous LLD	This Version
G1	**REVIEW missing from StepType**	*Feature table lists Review as a supported step type alongside Approval, Task, Checklist, Signature*	StepType enum omitted REVIEW entirely	REVIEW added to StepType with explanatory comment. Distinction from APPROVAL documented.
G2	**Action was not modelled as a type**	*AdvanceStep(instanceID, stepID, action Action, actor User) — Action is a named first-class type*	Action dissolved into ActionType enum + loose Map<String,Object> parameter	Action value object defined in Section 3. Static factory methods enforce valid construction. advanceStep now accepts Action.
G3	**StepResult was never defined**	*AdvanceStep returns StepResult*	StepResult referenced as return type but fields never documented	StepResult fully defined in Section 3 with all 7 fields, their types, and purpose documented.
G4	**In-memory vs persistent storage**	*Assignment states: in-memory data structures are sufficient*	StateStore introduced as persistent DB-backed	Deviation retained as intentional architectural upgrade. Documented

			component — richer than required	explicitly in ProcessEngine section with justification.
B1	**Regex compiled on every validate() call**	*Pattern objects are expensive. validate() is on the advanceStep hot path*	Pattern.compile(schema.regex()) called inside loop on every invocation — DefaultFieldValidator.java:49	FieldSchema now carries compiledPattern field. DefinitionValidator.compilePatterns() compiles once at registration. FieldValidator.checkRegexPatterns() calls compiledPattern.matcher() — zero Pattern.compile() on the hot path.
B2	**PENDING state was unreachable**	*PENDING and IN_PROGRESS are distinct. PENDING enables time-to-open SLA tracking and correct getRemainingParticipants() count*	activateStep created StepState directly as IN_PROGRESS — DefaultStepActivator.java:37	activateStep now creates PENDING. openStep() added to ProcessEngine public interface. TransitionGuard.assertStepIsInProgress() added. StepState gains createdAt and openedAt timestamps. ActionType.OPEN added.
B3	**Escalation event used wrong User constructor**	*User must carry a display name, not just an ID*	publishEscalationTriggered received new User(assignedTo, assignedTo) — ProcessEngine.java:195	UserResolver interface defined in Section 3. StepActivator.resolveAssignees() now returns List<User> via UserResolver. EscalationWorker resolves the target User before emitting the event. new User(id, id) eliminated from engine code.